

Mémo création carte Leaflet

Table des matières

1	vershtml.....	2
1.1	Arguments	2
1.2	Génération du source html	3
1.3	versgeojson.....	3
2	Création carte Leaflet.....	4
2.1	Colonnes spécifiques des vues 4Map.....	4
2.2	Afficher/masquer (griser) éléments de la légende à ouverture.....	4
2.3	Ordre des couches et libellés légende.....	5
2.4	Priorité d’affichage points, lignes, polygones	6
2.5	Ajustement taille bloc légende	6
2.6	Valeurs spécifiques traitées par le js	7
3	Debug	8
3.1	Origines des bugs	8
3.2	Types de bugs et action associée	8
3.3	Alertes auto.....	9
4	Exemples de génération	10
4.1	Lancement ps1.....	10
4.2	Lancement depuis pgadmin.....	10
4.3	Exemple d’une vue _4Map.....	10

1 vershtml

1.1 Arguments

Fonction postgres générant la carte Leaflet html

6 arguments dont 5 optionnels avec des valeurs par défaut

Renseigner obligatoirement '_nom_vue', format tableau de texte

Arguments				
	Type de données	Mode	Nom de l'argument	Valeur par défaut
	text[] ▾	IN ▾	_nom_vue	
	text ▾	IN ▾	_caseacoher	'0':text
	text ▾	IN ▾	_titre	'Carte DGAML':text
	boolean ▾	IN ▾	_clic_zoom_actif	true
	text ▾	IN ▾	_info_id	'vue_defaut':text
	text ▾	IN ▾	_rech_json	'false':text

- **_nom_vue** : tableau de vues, syntaxe SQL : vershtml_n (array['vue1','vue2', ...])
- Si **_caseacoher** n'est pas renseigné aucune couche n'est active à l'ouverture ('0' par défaut) : oeil barré et grisé dans sidebar. Si '1' => 1ere couche active, '1,1' : 2 1eres couches active, '1,0,1' : 1^{er} et 3eme couche actives. Si on veut juste la 5eme, mettre : '0,0,0,0,1'.
Concerne affichage des couches seulement pas les éléments légende (*)
- **_info_id** = valeur de la col 'id' de la table 'infos_carte' qui indique au code js de remplir le bloc Info par le contenu de la col. 'texte_info'. La f° ps1 'SI3P0-Ajouter-Information-Carte' remplit la table avec le texte info inclus dans le .ps1
- **_rech_json** est composé de 3 valeurs séparées par des « | » :
 1. n° couche à traiter = ordre de passage dans array['vue1', 'vue2', 'vue3', ...]
 2. filtre(s) de recherche = champ 'leafletsearch' ou nom de champ unique
 3. texte de l'invite de saisie, décrit l'objet recherché et les filtresOn peut mettre **plusieurs box** de recherche en séparant par des « ; ». Ex.: -
rechJson '1|leafletsearch|TMJA par RD, Année ; 4|RD |RD par n°'

(*) selon la config et les manips il peut donc y avoir des éléments affichés en double sur la carte => **veiller à ne pas afficher des doublons dès l'ouverture.**

1.2 Génération du source html

Ligne de génération complète en fin de f° vershtml_n :

`'return partieDébut || versgeojson_n (_nom_vue) || partieCodeJS'`

- 1ere partie : appels aux api js, css
- 2eme partie : couches geojson stockées dans un tableau js
- 3eme partie : cœur du code js

L'ensemble des fichiers js et css sont stockés en local. Cela évite les bugs suite à changements sur fichiers en ligne et permet les modifs local CD30. En contrepartie, on ne bénéficie pas d'éventuelles améliorations => vérifier ponctuellement évolutions. Ex. : version 2.0 Leaflet beta au 06/26 avec syntaxe js leaflet normalisée.

1.3 versgeojson

La fonction vershtml_n() appelle une autre f° versgeojson_n() qui génère les couches geojson issues des vues passées en param dans '_nom_vue' (tableau de vues).

- Conversion auto de tout format L93 en WGS84.
- Lignes et polygones transformés en ST_MultiLineString et ST_MultiPolygon
- Couches 2D/3D (3 coord x,y,z) passées en 2D via f° 'ST_Force2D'
- Lignes classées par ordre de grandeur décroissantes pour que les tronçons les plus petits soient en dernier dans le geojson donc dessinés en dernier, donc cliquables (et surlignables via switch) car situés par-dessus les plus grands.

2 Création carte Leaflet

Création de vues basées sur les tables/vues persistantes métiers du schéma « m ».

2.1 Colonnes spécifiques des vues 4Map

- Chaque vue doit obligatoirement contenir une colonne '**geom**' contenant des coord. Les élt sans coord sont ignorés mais ceux renseignés s'affichent
- Le js détecte les vues vides (0 ligne ramenée) ou sans aucunes coord. et les renomme/colore libellé en 'nom_vue vide'
- Le nom de la couche vient de la col. '**nomcouche**'. Si non renseigné, le js nomme en 'Couche 1', 'Couche 2' etc. Eviter d'utiliser des variables pr nom
- Pour les lignes, ajouter et remplir une colonne '**couleur**' dans la vue. Si colonne couleur absente, couleur par défaut = rouge
- Pour les points, ajouter et remplir une colonne '**icone**' dans la vue. Valeur : url vers l'icone. Si col. icone absente, icone par défaut = punaise bleue
- Pour ajouter une légende, ajouter et remplir une colonne '**legende**' dans la vue. Les valeurs constituent les libellés du bloc légende avec 1 valeur distincte.
- Ajouter col. bool '**dash**' et mettre à true pour afficher lignes en pointillés. Ex : 'case when traitement = 'Ponctuel' then true else null end as dash'=> seules lignes à traitement 'ponctuel' mises en pointillés, les autres en traits pleins
- Les autres col. de la vue formeront le contenu des popups. Les champs sont ordonnés dans la fenêtre par longueur de nom de champ croissante => dans la vue, jouer sur les espaces au nommage pour les classer selon besoins

Traitement js spécifique des champs et valeurs dans les popups :

- Champs « couleur | légende | icone | nomcouche | afficherlegouverture | leafletsearch | ordre_legende » exclus des popups et tooltips
- Valeurs de champs vides de la vue mis à « Non défini »
- Champs commençant par « Lien » formatés en lien http actif et exclus des tooltips. Mettre url visibles ds tooltips en nommant autrement le champ.

NB : 2 types de popups lignes identifiés par le js : 1 entité vs. n entités cad lignes superposées coord identiques mais avec 1 ou n valeurs de champs distinctes. Les points de coord identiques ont des popups distincts grâce au plugin spiderfy.

2.2 Afficher/masquer (griser) éléments de la légende à ouverture

Par rapport aux couches de la sidebar (icone oeil), la légende distingue les éléments d'une même couche avec une vue adaptée (besoin métier legende à étudier avant) :

- créer col. 'legende' contenant les libellés des légendes. A chaque libellé légende distinct doit correspondre un icone ou couleur spécifique.
- par défaut, avec 1 col. 'legende' et 1 col. 'icone' ou 'couleur' renseignés, les éléments seront actifs dans la légende à ouverture et affichés sur la carte
- pour griser une couche contenant 1 col. legende, ajouter 1 col. *'AfficherLegOuverture'* la mettre à 'false' ou vide. Réactiver en mettant à 'true'.
- pour griser une partie des éléments d'une couche, créer des filtres sql. Ex.
 - "CodeVigilanceHydro > 1 as AfficherLegOuverture" : type alerte >1 actives dans légende, celles = 1 grisées
 - "case when couleur = '#F29120' or couleur = '#449D67' then false else true end as AfficherLegOuverture" : 2 éléments spécifiques de la couche grisés dans légende à ouverture, les autres activés (else true). On aurait pu référencer col. 'legende' à la place de 'couleur' pour même résultat

Nombre d'éléments légende : le code js indique entre parenthèses le nombre d'éléments par couche dans la sidebar. Si besoin, pour avoir le nombre d'élts légende, modifier la col. légende, ex. : <nom_champ> || '(' || count(*) over (partition by <nom_champ>) || ')' as legende.

2.3 Ordre des couches et libellés légende

Ordre couches

Ordre de passage des vues/couches libre quel que soit leur type geom.

Les couches sidebar et bloc légende sont listées dans l'ordre de passage des vues.

Ordre libellés légende

- *Couche points* : les libellés légende suivent l'ordre défini dans la vue.
- *Couche lignes* : par défaut, les libellés d'une couche suivent le order by des tronçons desc de versgeojson remplaçant l'ordre défini dans la vue mais le js modifie l'ordre légende tout en respectant l'ordre d'affichage des tronçons sur la carte (tronçons petits par dessus tronçons plus longs) :
 - tri métier : créer col. num. *'ordre_legende'* dans la vue, l'alimenter des valeurs voulues (1,2,3...ordre libellés) puis faire un order by dessus.

Ex. : 'case CodeEtatAvancement3V when 1 then 'Tracé d'intention' when 2 then 'Etudes en cours' when 3 then 'Travaux en cours' when 4 then 'Ouvert' end as Legende, CodeEtatAvancement3V as ordre_legende , ... order by ordre_legende ' :



- si aucune col. 'ordre_légende' : le js classe par ordre alphabétique avec ordre intelligent des lignes (ex S1, S2, S10 au lieu de S1, S10, S2)
- éléments grisés placés par le js après les actifs par ordre alphabétique

NB : avec le tri alpha., utiliser quand ça fait sens (hiérarchie, classement), des astuces de nommage pour avoir l'ordre souhaité. Ex.: '1 – Niv. Structurant', '2 – Niv. de liaison', '3 – Niv. de proximité' sinon utiliser une col 'ordre_légende' plus neutre.

Doublons couleur/icône avec libellés légende distincts possibles dans une couche.

2.4 *Priorité d'affichage points, lignes, polygones*

A ouverture, pour même type geom, une couche active dans le bloc légende s'affiche par-dessus une active dans la sidebar (icône œil vert).

A ouverture pour même type geom à l'intérieur de la sidebar ou de la légende, priorité d'affichage par ordre des vues (1ère prioritaire sur les autres, ordre descendant).

- Priorité entre types geom à l'ouverture et en cours de consultation:
 - le js priorise toujours l'affichage des points (icônes) sur lignes/polygones
 - le js priorise toujours les lignes sur polygones. Ainsi lignes restent toujours cliquables sous un polygone même s'il vient d'être activé
- Priorité entre points :
 - à ouverture, le 1er élément légende s'affiche par-dessus les autres
 - en consultation, l'élément activé s'affiche par-dessus les autres
- Priorité entre lignes :
 - à ouverture, le dernier tronçon dessiné (du + grand au + petit cf. versgeojson) s'affiche par dessus les autres sur la carte (<> ordre légende cf. 2.3)
 - en consultation, l'élément activé s'affiche par dessus les autres

2.5 *Ajustement taille bloc légende*

Sur écran HD, pour éviter les débordements en hauteur, le bloc légende est via js :

- disposé sur 2 col. si nbr élts légende > 27

- réparti ensuite de façon équilibrée entre les 2 col.
- Conséquence : éviter dès la conception un nbre total d'éléments légende > 55 (lisibilité ?). Regrouper les éléments sur 1 thème pour diminuer leur nombre ou activer la couche entière dans sidebar si inutile de les distinguer 1 par 1

Remarques

- Le bloc légende est repliable en totalité ou par couches (chevrons)
- La taille de la fenêtre du navigateur joue aussi sur la bonne visibilité
- Le css revient à la ligne si libellé ou nom couche > 240px soit environ 50 caractères. Malgré cela, pour éviter de déborder en largeur et empiéter sur partie Est du Gard, **raccourcir** les libellés légende lors de la définition de la vue.
- Le plugin impression adapte carte et bloc selon hauteur légende (>400px): léger dezoom et décalage du Gard à gauche, diminution taille polices legende

2.6 Valeurs spécifiques traitées par le js

Selon la valeur de vue ou titre, le js va activer certaines parties de code :

- Si nom vue après normalisation est dans ["reseau routier", "reseau routier sanslegende", "lineaire", "utvi troncon", "utva troncon", "utal troncon", "utbe troncon", "utba troncon", "d30 troncon", « d30 3vitineraireroute »] + champ 'Niveau' pour couleur étiquette : déclenchement affichage dyn. étiquettes
- Titre contenant 'Surveillance du r' : activation widget météo, refresh toutes les 10 min., luminosité fond N&B atténué, heure de maj dans titre, icône Waze
- Titre 'Surveillance du r' ou 'Limites de gestion PER' (ou si sur 2 col pr 1 couche) : pas de nom couche dans bloc légende pour aérer, codé dans leaflet.legend.js
- Titre contenant 'fauchage' ou 'curage' : surlignage actif à ouverture
- Titre contenant 'UT de xxx' : centre la carte sur l'UT concernée à ouverture
- Titre commençant par 'Graflex' : centrage et zoom à ouverture, fond Ortho IGN
- Titre finissant par 3D : activation plugin visu bati 3D

3 Debug

Tester toute modification du source js d'abord en dur et à part sur le serveur [IIS qualif.](#)
Si modifs js, tester toute non régression : simuler manips. utilisateurs, vérif. plugins.

3.1 Origines des bugs

- suite à modif ou ajout de fonctionnalités dans vershtml pas ou mal testées
- suite à oubli ou mauvaise alimentation (valeurs, formats, doublons) des colonnes spécifiques des vues 4Map : geom, nomcouche, legende, ...
- suite à traitement de couches geojson contenant des valeurs ou incohérences jamais rencontrées jusque-là (*)
- autres : syntaxe script ps1 KO, modifs/perturbations sur SI CD30, SI tiers, etc.

(*) Le contenu 'anormal' d'une couche geojson s'explique par de nombreux facteurs : modifs effectuées via f° postgis avant transfo en geojson (phase alim), erreur/chgt de saisie à la source, évolution logicielle/API/formats, évolution du référentiel, des règlements... cette évolutivité doit aussi s'anticiper : prise d'info, newsletter, bot IA

3.2 Types de bugs et action associée

Carte ne se mettant pas ou plus à jour => après analyse des rapports d'erreur ps1:

- script sql de l'une des vues 4Map buggé => debug sous pgadmin
- script lancement ps1 buggé : attention aux caractères, tabulations de trop, retours lignes, C/C entre lignes de lancements mal adaptés

Carte plantée à ouverture ou après manipulations :

- Faire F12 console dans Edge pour identifier bug et localisation (n° ligne).
Rajouter ou décommenter des consoles.log, s'aider des commentaires js.
- Si non expert js, interroger la vue 4Map sous pgadmin ou isoler les couches geojson en début de code et analyser leur contenu sur des mots clés :
"coordinates", "type", "nomcouche", "legende", etc.

Une vue/couche ne s'alimente plus (ou n'a plus de coord):

- la couche est quand même indiquée en tant que 'couche n vide' dans box
- identifier l'origine du changement : source vide ou accès KO aux fichiers, logiciels, sites, API, .ps1/sql 'moissonner' d'alim. KO suite chgt formats, etc.

Le plantage d'une carte a « l'avantage » de soulever une anomalie sur les données reçues et d'identifier après analyse, les données à corriger.

Attention : un filtre SQL sur 1 couche geojson va débogger la carte mais aussi tronquer les infos visibles dessus => **corriger à la source en priorité**. Si impossible et qu'un patch SQL ou js est appliqué, vérifier au cours du temps, la cohérence des infos visibles sur carte ⇔ sources de données (tables teradata, autres) par rapport au contenu des vues et tableaux si3p0 liés.

3.3 Alertes auto

Prévoir un système d'alertes automatisées type envoi auto de mails en ps1 ou python sur les erreurs repérables :

- cartes dyn. à 0 ko après script quotidien
- cartes dyn. non mise à jour depuis x jours
- parsing des rapports d'erreur .txt et .err pour relevé d'anomalies par rapport à des mots clés type 'rollback'
- récupération des alertes issues de l'alimentation des tables teradata (timestamp ?)
- autres

4 Exemples de génération

4.1 Lancement ps1

Les noms des paramètres ps1 diffèrent légèrement des noms des arguments sql mais sollicitent bien leur équivalent pour générer une carte.

Ajout du paramètre '-carte' pour indiquer chemin et nom du fichier html de la carte

```
[void]$parametresJobs.Add((  
    Parametrer-Job-SI3P0-Generer-Carte `  
        -sources 'tmja_mensuel_global_4Map', 'ReseauRoutier_4Map', 'LimiteST_4Map'  
        -couchesActives 'LimiteST_4Map' `  
        -carte "$si3p0ThematiquesPortailWeb\Exploitation et trafic routier\Cartes  
        dynamiques\Trafic moyen journalier annuel (TMJA).html" `  
        -titre 'Trafic moyen journalier annuel (TMJA)' `  
        -idInfo 'MJA5AnsComptageRoutier' `  
        -rechJson '1|leafletsearch|TMJA par RD, Année, Commune, Type ; 2|RD |RD' `))
```

Attention : si on ajoute 1 paramètre de lancement, matcher l'ordre des paramètres de 'Parametrer-Job-SI3P0-Generer-Carte' ⇔ 'vershtml_n()' (contrainte ps1)

Si ps1 abandonné, créer l'équivalent de la f° « SI3P0-Generer-Carte » dans le langage concerné (python), recréer la connexion API à la base postgres et le mapping des arguments.

4.2 Lancement depuis pgadmin

Ordre de passage des arguments libres

```
select vershtml_n (array['structuresrh', 'entretien_utal', 'limitegestion_ut'],  
_caseacoher :='0,0,1', _rech_json:='1|Nom |structure RH par nom ;  
2|leafletsearch|activité par RD, prad, date', _titre:='test – fauchage UT Ales' ;
```

4.3 Exemple d'une vue _4Map

- 2 couches créées selon valeurs date récente/ancienne (cas particulier pour cette vue).
- Alertes waze grisées à ouverture dans la légende sauf accident, danger véhicule, inondation.
- Icones personnalisés attribués dynamiquement selon type d'alerte

```

create view AlerteWaze_4Map as
select
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alerte récente Waze' else 'Alerte ancienne Waze' end as "Nature ",
    to_char(a.DateCreation, 'DD/MM/YYYY HH24:mi') as "Date de création ",
    p._NumeroRoute as "RD",
    PRAEnTexte(p._PRA) as "PR+Abs",
    t.Description as "Type",
    st.Description as "Sous-type",
    a.Fiabilite as "Fiabilité",
    'https://www.waze.com/live-map?lat=' || round(ST Y(TransformerEnWGS84(a.Geom))::numeric, 6) || '&lon=' || round(ST X(TransformerEnWGS84(a.Geom))::numeric, 6) as "Lien LiveMap",
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alertes récentes Waze' else 'Alertes anciennes Waze' end as NomCouche,
    CASE
        WHEN t.description LIKE '%Accident%'
            OR st.description LIKE '%Accident%'
            OR st.description IN ('Danger présence véhicule urgence', 'Inondations') THEN true
        ELSE false
    END AS AfficherLegOuverture,
    case
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdTypeAlerteWaze || '.png'::varchar
        else '/Ressources/Images/Waze/70x70/couleur/MISC.png'::varchar
    end as Icone,
    case
        when a.IdSousTypeAlerteWaze <> '' then st.Description
        else t.Description
    end as Legende,
    a.Geom
from AlerteWaze a
cross join PointVersPRA(Geom, 25) p
inner join TypeAlerteWaze t on a.IdTypeAlerteWaze = t.IdTypeAlerteWaze
inner join TypeAlerteWaze st on a.IdSousTypeAlerteWaze = st.IdTypeAlerteWaze
where a.IdTypeAlerteWaze not in ('JAM', 'ROAD_CLOSED') and a.IdSousTypeAlerteWaze <> 'HAZARD_WEATHER_FOG'
and a.Fiabilite > 6;

```